

Instagram■Like Social Platform Production Architecture and

Operating Model Date: 2025-08-10 UTC

Instagram■Like Social Platform Production Architecture and

Operating Model Date: 2025-08-10 UTC

One■Pager Summary Goal Build a global photo and short■video social platform with fast creation and consumption, personalized feed and explore, real■time interactions, and strong safety and privacy. Deliver low latency, high availability, cost■aware scale, and an operationally excellent posture.

Key Outcomes

Sub three hundred millisecond p95 feed fetch and scroll. Media uploads that are immediately viewable with progressive processing. Highly engaging ranking and recommendations with guardrails. Safety at scale: moderation, abuse prevention, and privacy controls. Operational excellence: clear SLOs, canaries, and fast rollback.

Primary Assumptions

Tens to hundreds of millions of MAU globally. Peak read to write ratio around 20:1. Dominant traffic is media delivery from CDN; control plane is metadata heavy. Clients: iOS, Android, Web; regional data residency for some markets.

Functional Requirements

Core Create posts photos and short videos with captions, tags, mentions, and locations. Consume personalized Home feed, chronological Following, and Explore or Reels. Engage with likes, comments, saves, shares; follow or unfollow; hashtags and topics. Stories and ephemeral media; live streaming optional; direct messages separate service. Search for users, hashtags, places; notifications for activity. Safety and Trust Report and moderate content and accounts; spam and bot mitigation; age■gating and sensitive media controls. Creator and Business Insights dashboards, scheduling, link in bio, basic promotions ads optional. Admin Policy tooling, feature flags, traffic management, and A/B experiments.

Non■

Availability 99.95 percent read APIs, 99.9 percent write and upload APIs; media delivery 99.99 percent via CDN. Latency targets p95: feed page < 300 ms from edge; post write ACK < 200 ms; like or comment < 150 ms; search typeahead < 120 ms. Durability and integrity RPO 0 for accepted writes; strong consistency where correctness matters (follows, likes) with bounded eventual consistency for counters and denorms. Scalability horizontal across stateless services, caches, queues, and data stores; partition by user id and time.

Architecture Overview

Edge and Clients CDN terminates TLS 1.3 and serves media and static bundles. Edge functions handle A or B bucketing, device hints, and signed upload URLs. Mobile apps use gRPC or REST over HTTP 2 or 3; realtime via WebSocket or MQTT for notifications. Control Plane Services Auth and profiles, social graph, post service, comments, reactions, notifications, search, ranking, explore, stories, live metadata, abuse and moderation, ads optional, experiment config, feature flags. Media Pipeline Uploads go directly to object storage via pre■signed URLs. A media processor consumes events to transcode, generate thumbnails, keyframes, and story sprites; writes variants and manifests; signals readiness. Feed and Timeline Hybrid fan■out: fan■out to online followers and to per user home cache; on demand ranking with features from content, user, and graph stores. Explore is pull■based with heavy ranking and near real time freshness. Streaming Backbone Kafka or Kinesis for events: post created, reaction added, follow edge, moderation decisions, media ready, counters, and analytics taps. Serving and Caching Redis Cluster for hot timelines, entity caches, and counters. CDN plus edge caches for media. Application■side LRU for denormed records. Search and Discovery OpenSearch for users, hashtags, and places; vector search optional for semantic discovery. Analytics ClickHouse or BigQuery or Snowflake over S3 or GCS lake (Parquet with Iceberg or Delta). Real■time dashboards for KPIs and

experiments.

Data Flow (Happy Paths) Create Post Client requests pre-signed URL → uploads media to object storage → calls CreatePost with metadata → Post service writes metadata and emits post_created → Media pipeline processes, emits media_ready → Feed fanout and home cache warmup. Fetch Home Client → Gateway → Feed service gets candidates from home cache or fanout store → Ranking service scores with features and returns page → Client prefetches next page and media variants from CDN. Engage Like or comment writes to transactionally safe store with outbox; counters update asynchronously; notifications fanout; abuse models scored.

Technology Stack (reference) Edge and Gateway CDN, WAF, API Gateway, OAuth 2.1 or OpenID Connect, mTLS in mesh. Compute Kubernetes with autoscaling; service mesh with workload identity; gRPC for internal calls. Caches Redis Cluster for timelines, sessions, feature caches, rate limits. OLTP and Graph User and social graph: sharded Postgres or MySQL for consistency; follower edges also in Cassandra or Scylla for high fanout reads. Posts and reactions: Postgres sharded and replicated; outbox pattern for Kafka. Timelines Cassandra or Scylla or DynamoDB for per user feeds and story indexes; Redis for hot pages. Search and Explore OpenSearch plus vector index (FAISS or pgvector or OpenSearch kNN). Media Object storage (S3 or GCS) with lifecycle; transcoding on GPU or accelerated nodes; HLS or DASH manifests for video. Observability OpenTelemetry, Prometheus and Grafana, Loki, Jaeger or Tempo; SIEM for security logs. CI and CD GitHub or GitLab, Terraform or Pulumi, Helm, Argo CD and Argo Rollouts; SAST or DAST; signed images and SBOM.

Storage Strategy

Hot Redis for home cache pages, entity cache, counters; TTL tuned per surface. Region-local replicas. Warm Cassandra or DynamoDB for timelines and story indexes; sharded relational for posts, reactions, comments, follows. Cold Object storage for media and logs; lakehouse for analytics with compaction and clustering. Indexing Denormalize popular fields (author handle, avatar, counts) into timeline rows for fewer lookups; write via change data capture and materialized views. Scaling Approach Sharding and Keys Partition users and posts by user_id hash and time buckets; use consistent hashing for timelines. For superstars, apply write gate and fanout throttling, or use pull-heavy model for their followers. Fanout Strategy Fanout on write to online followers, fanout on read for cold followers; decay old entries; maintain backfills for late joins. Ranking Feature store caches recent features. Rankers are versioned and deployed behind AB layers; models run on CPU first, GPU for heavy Explore. Backpressure and Degradation During surges drop non critical counters, reduce media quality tiers, fall back to chronological feed; pause Explore refresh; queue writes with TTL.

Data Models

(abridged) User user_id, handle, name, bio, avatar, privacy flags, created_at, settings. FollowEdge follower_id, followee_id, created_at, status. Post post_id, author_id, media_refs[], caption, tags[], place_id, visibility, created_at, edited_at, status. Story story_id, author_id, media_ref, expires_at, viewers_count. Reaction post_id, user_id, type like or save, created_at. Comment comment_id, post_id, user_id, body, parent_id, created_at, status. TimelineEntry user_id, post_id, score, source reason follow or explore, created_at. MediaVariant post_id, variant_id, codec, bitrate, width, height, duration, ready_at. ModerationCase entity_type, entity_id, reason, state, actions, created_at, updated_at.

Latency Budgets and SLAs

Gateway auth and routing under 20 ms p95. Home page fetch server time under 120 ms p95; end to end under 300 ms p95 from edge. CreatePost metadata write under 100 ms p95; media ready target under 2 minutes for HD, < 10 seconds for mobile preview. Like or comment ACK under 150 ms p95. Search typeahead under 120 ms p95. Availability read 99.95 percent; write 99.9 percent; media delivery 99.99 percent via CDN.

Key Algorithms and Workflows

Dedup and Idempotency Idempotency keys for post and comment creates; outbox tables publish to Kafka atomically with DB commit. Feed Generation Candidate sets from author

follow graph and recent post windows; remove seen posts; heavy rankers blend freshness, social

proximity, quality signals, safety; diversity constraints avoid monotony. Explore Near real time embeddings or topic indexes; retrieve top candidates per interest; re-rank with popularity and session-level feedback; safe search filters. Spam and Abuse Rate limits, device and IP reputation, link and mention heuristics, graph-based spam detection; ML for account integrity; shadow limits; appeals workflow. Media QoE Adaptive bitrate; keyframe aligned GOP; per region CDNs selection; partial cache fill with range requests; prefetch heuristics on scroll. Consistency Counters are eventually consistent and reconciled with periodic jobs; critical state (follows, privacy) strongly consistent with transactions.

Security, Reliability, Compliance Security TLS 1.3, HSTS, CSP; OAuth scopes, RBAC and ABAC; mTLS in mesh; KMS envelope encryption; signed webhooks; strict input validation; SSRF and egress allow lists. Privacy Minimize PII; data subject APIs for access, export, delete; audience controls; child safety features; residency routing by tenant or region; retention policies. Reliability Multi-AZ cells; active-active for stateless tiers; quorum for stateful; chaos and DR drills; error budgets; canaries and automatic rollback. Compliance SOC 2 Type 2; GDPR and CCPA; vendor DPAs; audit trails.

Multi-Tenancy Tenant id in every key; per tenant quotas, rate limits, encryption keys, and dashboards; optional hard isolation with dedicated cells and VPCs for brands or regions; feature flags per tenant. CI CD and Release Safety Everything as code; progressive delivery with Argo Rollouts; automated rollback on SLO burn or anomaly; schema registry for events; contract tests; online DDL with shadow reads; SBOM and signed images.

APIs and Integration Points Public POST create_post; GET feed; GET explore; POST like; POST comment; POST follow or unfollow; GET search; GET stories; POST story; GET notifications. Partner Moderation queues and review tools; ads or promotions; analytics export; webhooks for business accounts. Internal Feature store, ranker gRPC, social graph, media processor, experiment service, push notifications.

Observability and Operations

Golden dashboards Feed p95 and p99, cache hit ratio, Kafka lag, timeline write latency, media processing backlog, CDN hit ratio, error budget burn, 95th scroll stalls. Tracing and logs End-to-end traces include user id hash, request id, experiment cohort; PII scrubbing. Runbooks Hot shard relief, superstar fan-out throttle, ranking rollback, media backlog drain, regional failover, cache cold start.

Capacity Planning and Cost Estimate (illustrative per busy region) Traffic browse 50k-200k RPS; write 3k-10k RPS; events 100k-300k EPS; media egress dominates cost. Kafka 6-12 brokers RF=3; partitions sized to hot topics. Redis 12-24 shards target < 65 percent memory. Cassandra or DynamoDB sized for feed QPS with 30 percent headroom. Relational cluster 1 writer + 3-5 replicas per shard; online re-sharding plan. Estimated monthly infra 80k-250k per busy region depending on media egress, retention, and search scale.

Testing Strategy and Failure Drills

Unit and property tests for ranking features and counters; contract tests for APIs and event schemas. Load and soak for feed, post create, comments; chaos for broker loss, shard failover, CDN vendor failover, superstar load, and regional failover. Disaster recovery drills for DB restore, media index rebuild, and object store version restore; synthetic traffic for SLO validation.

Phased Build Plan

MVP three months Auth, profiles, create and view posts with captions, likes and basic comments, home feed chronological, media uploads with preview transcode, notifications, baseline observability. Phase two six to nine months Personalized feed and explore, hashtags and search, stories, moderation tooling, canary deployment, analytics marts, multi-region cells. Phase three twelve months Advanced ranking, vector search explore, live and reels, hard tenant isolation options, cost optimizations, stricter SLOs and resilience playbooks.

Key Choices Trade Offs Risks and Mitigations Hybrid fan-out balances immediacy and cost; adjust

thresholds by author size and follower freshness. Cassandra or DynamoDB timelines reduce tail latency but add operational complexity; mitigate with automation and per partition hygiene. Strong consistency on follows and privacy increases write cost; mitigate with careful sharding and short transactions. Ranking gains engagement but risks filter bubbles; enforce diversity constraints and safety filters; provide user controls. Media egress drives cost; maximize CDN hit ratio, aggressive caching, and lifecycle policies.